

**Министерство науки и высшего образования Российской  
Федерации** ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО  
ОБРАЗОВАНИЯ «**Национальный исследовательский университет ИТМО**»  
(Университет ИТМО)

**Факультет прикладной информатики**  
**Образовательная программа** Мобильные и сетевые технологии  
**Направление подготовки** 09.03.03 Прикладная информатика

**О Т Ч Е Т**  
по курсовой работе дисциплины Мобильная разработка (Android и IOS) на  
тему  
**“Разработка Android-приложения Мессенджер”**

**Обучающийся:**  
Петухов Семён Алексеевич, К3439

**Преподаватель:**  
Шатинский Григорий Сергеевич

Санкт-Петербург  
2025

## Оглавление

|                                                                                |                                        |
|--------------------------------------------------------------------------------|----------------------------------------|
| 1. Введение.....                                                               | 3                                      |
| 3. Архитектура системы .....                                                   | 4                                      |
| 4. Реализация функционала приложения .....                                     | 7                                      |
| 4.1. Навигация между экранами через Navigation Graph и Bottom Navigation ..... | 7                                      |
| 4.2. Экран профиля: редактирование и сохранение имени и статуса.               | 10                                     |
| 4.3. Экран настроек: переключатель темы (Dark/Light Mode) .....                | 13                                     |
| 4.4. Экран ленты сообщений: загрузка из API, отображение и офлайн-доступ ..... | 16                                     |
| 4.5. Уведомления о новых сообщениях через WorkManager и Notification.....      | 21                                     |
| 4.6. Разрешения при запуске, реагирование на онлайн/офлайн-события .....       | <b>Ошибка! Закладка не определена.</b> |
| 5. Основные фрагменты кода с пояснениями                                       | <b>Ошибка! Закладка не определена.</b> |
| 6. Выводы.....                                                                 | 25                                     |

## **1. Введение**

Данная курсовая работа посвящена разработке мобильного приложения под Android.

**Цель** – получить навыки создания Android-приложения с использованием современных технологий мобильной разработки (Android Jetpack, архитектура MVVM). Разрабатываемое приложение должно имитировать клиент мессенджера: пользователь может редактировать свой профиль, переключать тему оформления, просматривать ленту сообщений, получать уведомления о новых сообщениях.

Приложение разработано в среде Android Studio с использованием языка Kotlin. В процессе реализации используются компоненты Android Jetpack (LiveData, ViewModel, Navigation, Room, WorkManager и др.), а также библиотека Retrofit для сетевого взаимодействия.

### **Задачи**

- 1) Реализовать архитектуру приложения на основе MVVM.
- 2) Настроить навигацию между экранами (профиль, настройки, лента сообщений) через Navigation Component и BottomNavigationView.
- 3) Реализовать экран профиля с возможностью редактирования и сохранения имени и статуса пользователя во ViewModel.
- 4) Реализовать экран настроек с переключателем светлой и тёмной темы, состояние темы сохраняется во ViewModel.
- 5) Реализовать экран ленты сообщений: отображение списка сообщений через RecyclerView, загрузка данных из API с использованием Retrofit и сохранение их в локальную базу Room для офлайн-доступа.

6) Реализовать периодическую фоновую синхронизацию сообщений с помощью WorkManager, отображать уведомление при получении новых данных.

7) Обеспечить корректный запрос разрешения на отправку уведомлений при запуске приложения (Android 13+).

8) Реализовать обработку состояния подключения к сети: при отсутствии интернета использовать локальные данные, отображать состояние подключения.

### 3. Архитектура системы

В приложении реализована архитектура **MVVM (Model–View–ViewModel)** – современный шаблон проектирования, рекомендованный Google для Android-приложений. MVVM обеспечивает разделение ответственностей между слоями приложения: пользовательский интерфейс (View) отделён от логики и управления данными (ViewModel и Model). Это повышает поддерживаемость кода и облегчает тестирование и расширение функционала.

**Model** - отвечает за управление данными приложения. Это включает как удалённые источники данных (веб-сервис API для сообщений), так и локальные источники (база данных на устройстве).

**ViewModel** - представляет слой представления данных и логики на стороне клиента. Каждый экран (например, профиль, настройки, лента сообщений) имеет свой ViewModel, который хранит состояние UI и выполняет необходимые действия. ViewModel получает данные из Repository и обеспечивает их кэширование для UI. Главное преимущество ViewModel – сохранение состояния при изменениях конфигурации (например, поворот экрана). ViewModel выступает «мостом» между View и Model: он запрашивает и обновляет данные в Model, и предоставляет данные View через наблюдаемые объекты.

**View** – слой пользовательского интерфейса, представленный Activity и Fragment. В нашем приложении каждый раздел (профиль, настройки, лента) реализован как Fragment, встроенный в основную Activity. View отвечает за

отображение данных и обработку взаимодействия пользователя (нажатия кнопок, ввод текста). Код UI не содержит бизнес-логики: вместо этого, UI обращается к своему ViewModel за данными и отправляет пользовательские события (например, запрос сохранить профиль) во ViewModel. Благодаря этому достигается слабая связанность.

На диаграмме ниже представлена общая схема архитектуры приложения, демонстрирующая основные компоненты и взаимодействие между ними:

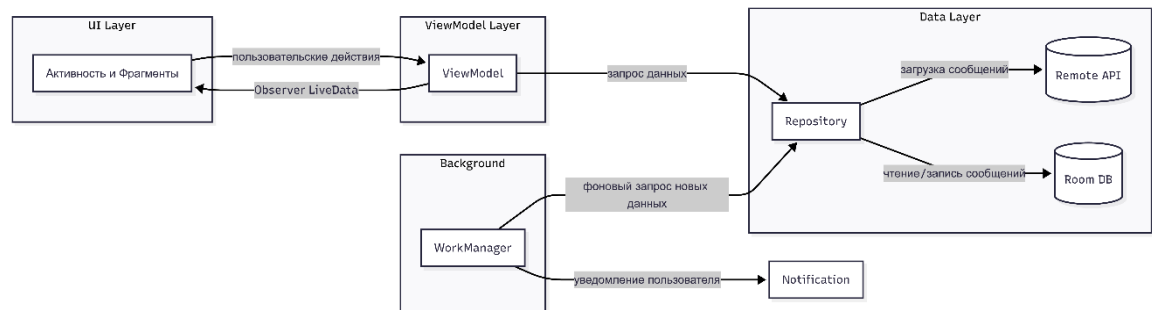


Рисунок 1 – общая структура системы

### Используемые компоненты архитектуры (Jetpack):

**LiveData.** Для хранения и оповещения об изменении данных используется LiveData – это класс-хранилище данных, поддерживающий механизм наблюдения и учитывающий жизненный цикл компонентов UI. Например, в ProfileViewModel имя и статус пользователя представлены как LiveData<string>: экран профиля подписывается (observe) на эти объекты и автоматически получает обновления при изменении данных. Благодаря LiveData, UI всегда актуален по отношению к данным, и нет необходимости вручную опрашивать ViewModel – достаточно один раз установить наблюдателя.

**ViewModel.** Класс ViewModel (как описано выше) предоставляет хранение состояния экрана и бизнес-логику, привязанную к жизненному циклу UI. ViewModel сохраняется в памяти, пока соответствующий экран активен, и переживает поворот экрана, что устраняет необходимость повторного запроса данных при конфигурационных изменениях. В проекте для каждого экрана создан свой ViewModel: ProfileViewModel, SettingsViewModel, FeedViewModel и

т.д. Они наследуют от базового ViewModel и инициализируются scope`ом жизненного цикла фрагмента. Также при необходимости используется SavedStateHandle для сохранения данных ViewModel при принудительном уничтожении процесса – это обеспечивает восстановление критичных данных (например, выбранной темы) при перезапуске приложения.

**Room.** Для локального хранилища данных используется библиотека **Room** (модуль Jetpack Persistence). Room предоставляет объектно-ориентированный слой поверх SQLite, позволяя удобно сохранять и извлекать объекты Kotlin/Java из базы данных. В нашем приложении Room используется для кэширования сообщений: загруженные из интернета сообщения сохраняются в БД, чтобы пользователь мог просматривать ленту офлайн

**Retrofit.** Для сетевого взаимодействия выбран популярный клиент Retrofit. Он упрощает выполнение HTTP-запросов. В проекте описан интерфейс ApiService с методами для получения ленты сообщений. Retrofit автоматически преобразует JSON-ответ сервера в список объектов Message

**WorkManager.** Для фоновых задач, которые должны выполняться даже при неактивном приложении, используется WorkManager – библиотека Jetpack для отложенных и периодических работ. WorkManager гарантирует выполнение задач при соблюдении заданных условий и сохраняет задачи между перезапусками приложения и перезагрузками устройства. В проекте WorkManager отвечает за периодическую обновление сообщений и оповещение об этом пользователя

**Navigation Component.** Для организации переходов между экранами применён Android Navigation Component. Он использует Navigation Graph – описанный в XML граф экранов приложения и связей между ними. В нашей навигационной схеме определены три основные вкладки (профиль, настройки, лента) как фрагменты на нижней панели навигации. Был применён

BottomNavigationView, связанный с NavController, так реализована навигация по нажатию на иконки в нижней части меню.

**Другие компоненты.** Также в проекте используются:

**RecyclerView** для эффективного отображения списка сообщений в ленте

**Material Design компоненты** (например, TextInputLayout, Buttons, SwitchMaterial для переключателя темы) для современного внешнего вида приложения.

**Notification API** для создания системных уведомлений о новых сообщениях

**ConnectivityManager** (вместе с **NetworkCallback**) для отслеживания состояния сети – чтобы приложение могло реагировать на переход в офлайн/онлайн

Таким образом, архитектура приложения соответствует современным требованиям: MVVM обеспечил разделение логики, Jetpack-компоненты упростили реализацию сложных функций (навигации, хранения данных, фоновой работы).

#### **4. Реализация функционала приложения**

В данном разделе подробно описана реализация ключевых функциональных возможностей приложения

##### **4.1. Навигация между экранами через Navigation Graph и Bottom Navigation**

Навигация в приложении организована с помощью Navigation Component. Был создан файл графа навигации nav\_graph.xml, в котором определены три главных фрагмента приложения:

- *ProfileFragment* – экран профиля пользователя.
- *SettingsFragment* - экран настроек.
- *FeedFragment* – экран ленты сообщений.

Эти три экрана обозначены как top-level destinations и связаны с элементами BottomNavigationView – нижней навигационной панели. В разметке главной

Activity (*activity\_main.xml*) присутствует компонент `BottomNavigationView` с тремя пунктами меню (Профиль, Настройки, Лента), а также контейнер `NavHostFragment`, в котором будет отображаться текущий экран. Фрагменты привязаны к пунктам меню через соответствующие идентификаторы (`id`). Это позволяет `NavigationUI` автоматически обрабатывать выбор пункта меню: при нажатии на иконку панель сама вызывает `NavController.navigate()` к соответствующему фрагменту, а также обновляет отметку выбранного элемента в интерфейсе.

Основная Activity выполняет инициализацию навигации в `onCreate()`:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        // Получаем NavController из хоста
        val navHostFragment =
            supportFragmentManager.findFragmentById(R.id.nav_host_fragment)
                                   as NavHostFragment
        val navController = navHostFragment.navController
        // Настраиваем BottomNavigationView с NavController
        val bottomNav =
            findViewById<BottomNavigationView>(R.id.bottomNavigationView)
            bottomNav.setupWithNavController(navController)
    }
}
```

Этот код настраивает навигацию: из макета извлекается `NavHostFragment` и `BottomNavigationView`, после чего методом `setupWithNavController` они соединяются. Таким образом, навигация между экранами выполняется без дополнительного кода при кликах по пунктам меню.



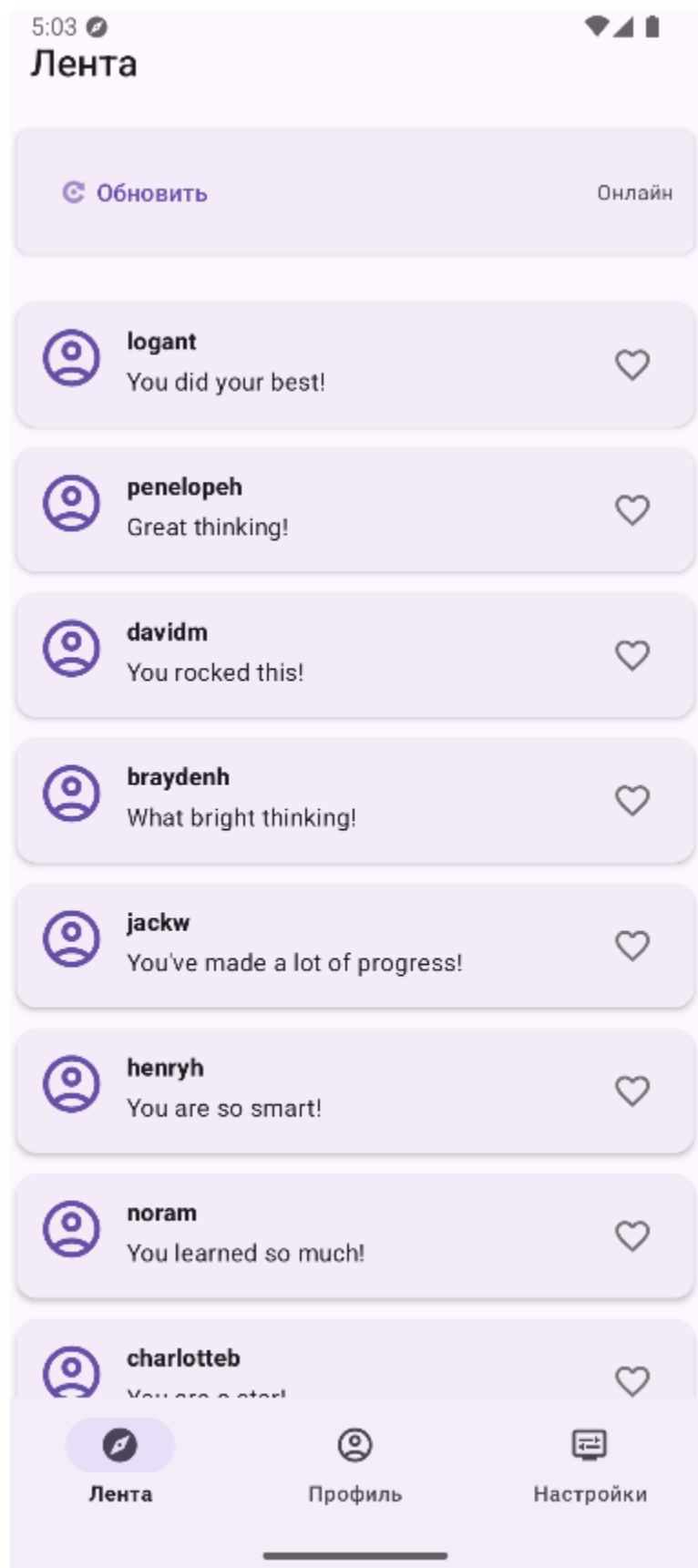


Рисунок 2 - Лента экран с Bottom Navigation

## 4.2. Экран профиля: редактирование и сохранение имени и статуса

**Экран профиля** предоставляет пользователю интерфейс для просмотра и изменения персональных данных — имени и статуса. Он реализован в ProfileFragment (с макетом fragment\_profile.xml). В макете расположены, например, EditText для ввода имени, EditText для ввода статуса и кнопка "Сохранить"

Логика экрана сосредоточена во **ViewModel профиля** (ProfileViewModel). При открытии фрагмента, ProfileViewModel загружается (он привязан к жизненному циклу фрагмента).

В ProfileViewModel определены поля:

```
class ProfileViewModel : ViewModel() {
    val userName: MutableLiveData<String> = MutableLiveData("")
    val userStatus: MutableLiveData<String> = MutableLiveData("")

    fun updateProfile(name: String, status: String) {
        userName.value = name
        userStatus.value = status
    }
}
```

Здесь userName и userStatus — объекты LiveData для имени и статуса. Инициализируем их пустыми строками. Метод updateProfile обновляет эти LiveData новыми значениями, когда пользователь сохраняет изменения.

Во фрагменте профиля мы наблюдаем за LiveData из ViewModel, чтобы автоматически заполнить UI актуальными данными:

```
class ProfileFragment : Fragment() {
    private val viewModel: ProfileViewModel by viewModels()

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        // Подписка на имя пользователя
        viewModel.userName.observe(viewLifecycleOwner) { name ->
            nameEditText.setText(name)
        }
        // Подписка на статус пользователя
        viewModel.userStatus.observe(viewLifecycleOwner) { status ->
```

```
        statusEditText.setText(status)
    }
    // Обработка нажатия на кнопку "Сохранить"
    saveButton.setOnClickListener {
        val newName = nameEditText.text.toString()
        val newStatus = statusEditText.text.toString()
        viewModel.updateProfile(newName, newStatus)
        Toast.makeText(requireContext(), "Профиль сохранён",
Toast.LENGTH_SHORT).show()
    }
}
}
```

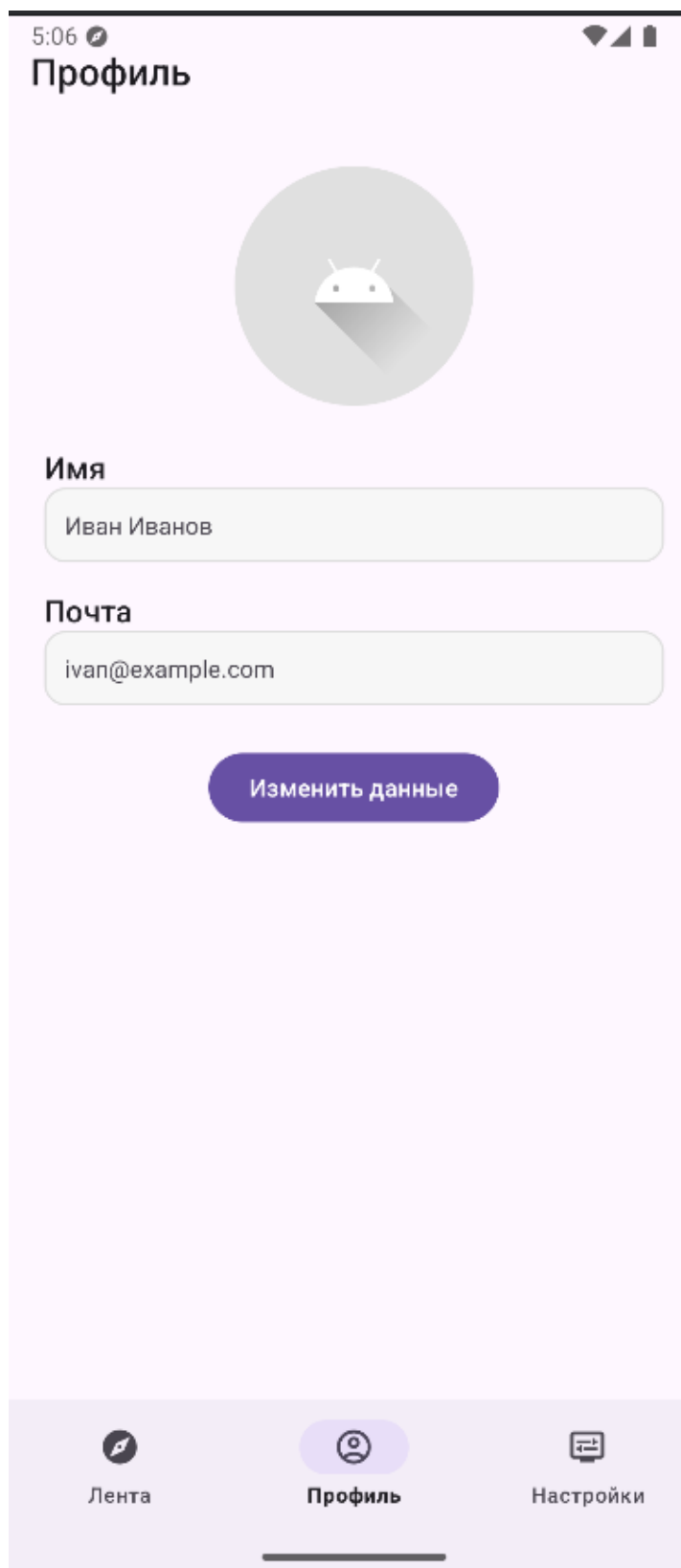


Рисунок 3 - Экран профиля

### 4.3. Экран настроек: переключатель темы (Dark/Light Mode)

Экран настроек (SettingsFragment) содержит опцию переключения темы приложения для выбора между светлой и тёмной темой оформления. Эта функциональность позволяет пользователю настроить внешний вид приложения под свои предпочтения.

Реализация темы основана на Android DayNight Theme. В стилях приложения изначально подключена тема, поддерживающая переключение (например, Theme.MaterialComponents.DayNight). Переключение темы обычно достигается вызовом метода:

```
AppCompatDelegate.setDefaultNightMode(mode)
```

где mode – это MODE\_NIGHT\_YES (тёмная) или MODE\_NIGHT\_NO (светлая). В нашем случае, мы используем Switch, при изменении положения которого вызываем этот метод.

Для хранения состояния переключателя используется **SettingsViewModel**. В нём заведено поле, отражающее текущую выбранную тему:

```
class SettingsViewModel : ViewModel() {  
    val darkThemeEnabled: MutableLiveData<Boolean> = MutableLiveData(false)  
}
```

По умолчанию значение false – то есть светлая тема. Если пользователь переключил тумблер, мы обновляем это LiveData:

```
// В SettingsFragment  
themeSwitch.setOnCheckedChangeListener { _, isChecked ->  
    viewModel.darkThemeEnabled.value = isChecked  
    val mode = if (isChecked)  
        AppCompatDelegate.MODE_NIGHT_YES  
    else  
        AppCompatDelegate.MODE_NIGHT_NO  
    AppCompatDelegate.setDefaultNightMode(mode)  
}
```

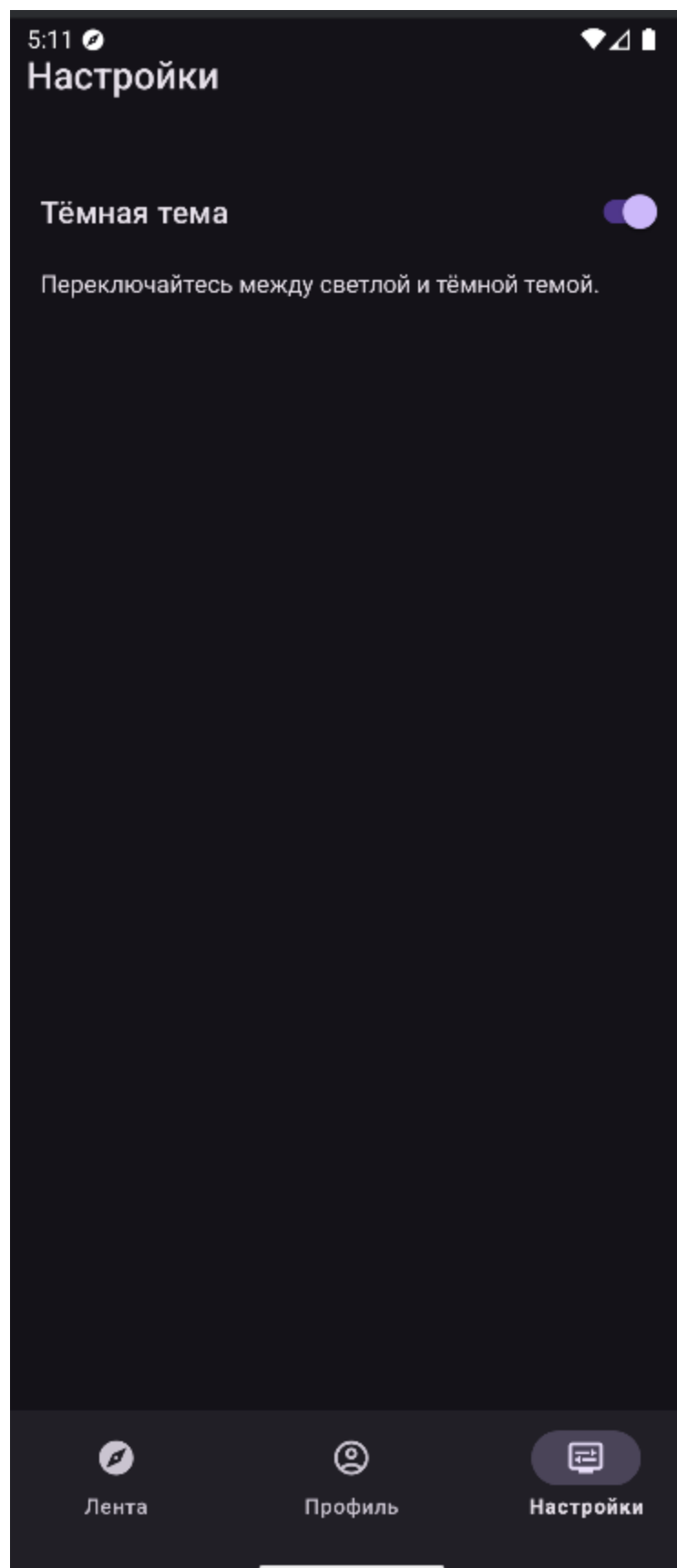
Кроме того, при создании фрагмента настроек мы подписываемся на darkThemeEnabled:

```
viewModel.darkThemeEnabled.observe(viewLifecycleOwner) { isDark ->
```

```
themeSwitch.isChecked = isDark  
}
```

Это позволяет, при открытии экрана, установить переключатель в правильную позицию (если тема уже была ранее выбрана, ViewModel хранит состояние).

После перезапуска UI применит новую тему: это заметно по изменению цветов фона, текста и компонентов интерфейса.



**Рисунок 4 -Экран настроек с переключателем темы**

#### 4.4. Экран ленты сообщений: загрузка из API, отображение и офлайн-доступ

Экран ленты сообщений (FeedFragment) является центральной частью приложения, он отображает список сообщений. Этот экран демонстрирует работу с сетью (получение данных с сервера), с локальной базой (кэширование) и отображение списка с помощью RecyclerView.

**Получение данных из API (Retrofit).** В приложении настроен сетевой клиент Retrofit для общения с удалённым сервером. Для интеграции мы:

- Добавили зависимости Retrofit и конвертера Gson.
- Описали интерфейс ApiService:

```
interface ApiService {  
    @GET("messages")  
    suspend fun getMessages(): List<Message>  
}
```

- Создали экземпляр Retrofit в Singleton-объекте (например, RetrofitClient) с указанием базового URL и конвертера Gson:

```
val retrofit = Retrofit.Builder()  
    .baseUrl("https://example.com/api/")  
    .addConverterFactory(GsonConverterFactory.create())  
    .build()  
val apiService = retrofit.create(ApiService::class.java)
```

Этот apiService далее используется в Repository для вызова getMessages().

**Локальное хранилище (Room).** Настроена база данных Room с сущностью Message. Класс Message помечен @Entity, имеет поля (id, text, author, timestamp и др.). Интерфейс MessageDao содержит методы:

```
@Dao  
interface MessageDao {  
    @Query("SELECT * FROM Message ORDER BY timestamp DESC")  
    fun getAllMessages(): LiveData<List<Message>>  
  
    @Insert(onConflict = OnConflictStrategy.REPLACE)  
    suspend fun insertMessages(messages: List<Message>)  
}
```



**Репозиторий сообщений.** Создаём **MessageRepository**, который инкапсулирует логику получения сообщений из сети и базы:

```
class MessageRepository(  
    private val api: ApiService,  
    private val dao: MessageDao  
) {  
    // LiveData из базы, сразу доступная для UI  
    val allMessages: LiveData<List<Message>> = dao.getAllMessages()  
  
    suspend fun refreshMessages() {  
        try {  
            val freshMessages = api.getMessages() // сетевой запрос  
            dao.insertMessages(freshMessages)      // кэширование в базе  
        } catch (e: Exception) {  
            // Обработка ошибки сети (например, логирование)  
        }  
    }  
}
```

При вызове `refreshMessages()` пытаемся загрузить данные из интернета. В случае успеха – сохраняем/обновляем их в локальной базе. Если произошла ошибка (нет соединения или сервер недоступен), метод ловит `Exception`. Поле `allMessages` – `LiveData`, сразу полученное из Room DAO при инициализации репозитория; оно всегда содержит актуальный кэш сообщений и обновляется автоматически, когда вызывается `insertMessages` (Room при работе с `LiveData` сам уведомит наблюдателей об изменении данных в таблице). Это позволяет реализовать `offline-first` подход: UI фрагмента подписан на `LiveData` из базы и всегда показывает локальные данные, а репозиторий в фоновом потоке обновляет базу из сети. Если сеть недоступна, UI просто покажет старые сообщения из базы (пользователь все равно увидит контент).

**ViewModel ленты (FeedViewModel).** В `FeedViewModel` мы храним `LiveData` списка сообщений и вызываем обновление данных:

```
class FeedViewModel(application: Application) : AndroidViewModel(application) {  
    private val repository: MessageRepository  
    val messages: LiveData<List<Message>>  
  
    init {
```

```

        // Инициализируем БД и репозиторий (упрощённо, можно через DI)
        val dao = AppDatabase.getDatabase(application).messageDao()
        val api = ApiClient.apiService // статический доступ к инициализированному
Retrofit API
        repository = MessageRepository(api, dao)
        messages = repository.allMessages
        // Запускаем начальную загрузку
        viewModelScope.launch {
            repository.refreshMessages()
        }
    }

    fun refresh() {
        viewModelScope.launch {
            repository.refreshMessages()
        }
    }
}

```

Здесь FeedViewModel наследует AndroidViewModel, чтобы иметь контекст для инициализации базы. При создании ViewModel сразу получаем LiveData из репозитория (подписка на базу) и иницилируем загрузку из сети (через корутину)

В коде также определён метод refresh(), позволяющий вручную обновить ленту — его можно вызывать, например, при жесте свайпа-вниз (pull-to-refresh) или при повторном открытии экрана, чтобы обновить данные.

**Отображение в UI (RecyclerView).** В макете fragment\_feed.xml содержится **RecyclerView** для списка сообщений. Мы создали адаптер MessageAdapter, который заполняет элементы списка. В FeedFragment при создании view мы настраиваем RecyclerView:

```

class FeedFragment : Fragment() {
    private val viewModel: FeedViewModel by viewModels()
    private lateinit var adapter: MessageAdapter

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        adapter = MessageAdapter()
        recyclerView.adapter = adapter
        recyclerView.layoutManager = LinearLayoutManager(requireContext())
        // Подписываемся на LiveData списка сообщений
    }
}

```

```

viewModel.messages.observe(viewLifecycleOwner) { messageList ->
    adapter.submitList(messageList)
}
// Реализуем pull-to-refresh через SwipeRefreshLayout
swipeRefreshLayout.setOnRefreshListener {
    viewModel.refresh()
}
// Отключаем индикатор refresh по окончании обновления
viewModel.messages.observe(viewLifecycleOwner) {
    swipeRefreshLayout.isRefreshing = false
    // (В реальном коде лучше разделить статус загрузки и данные)
}
}
}

```

Также добавлен элемент **SwipeRefreshLayout** для удобного обновления свайпом. При его срабатывании вызываем `viewModel.refresh()`, что инициирует корутину с загрузкой из сети. Когда LiveData сообщений меняется (что происходит после вставки в базу), мы отключаем крутящийся индикатор (`isRefreshing = false`).

**Offline-функциональность:** Если приложение запускается без интернет-соединения, то `repository.refreshMessages()` бросит исключение в блоке `try-catch`, и база не обновится. Однако, `FeedViewModel.messages` уже содержит LiveData из базы – если в базе остались данные с прошлого сеанса, они будут показаны. Таким образом, пользователь увидит последнюю доступную ленту сообщений даже офлайн. Если же пользователь первый раз запускает приложение офлайн и база пуста, то список будет пустым.

**Работа с потоками:** Загрузка из сети и запись в базу выполняются в `viewModelScope.launch`, то есть асинхронно на IO-диспетчере.

В результате, экран ленты отображает всегда актуальные данные при наличии сети, а при её отсутствии – последний кэш. Это соответствует требованиям на устойчивость к соединению.

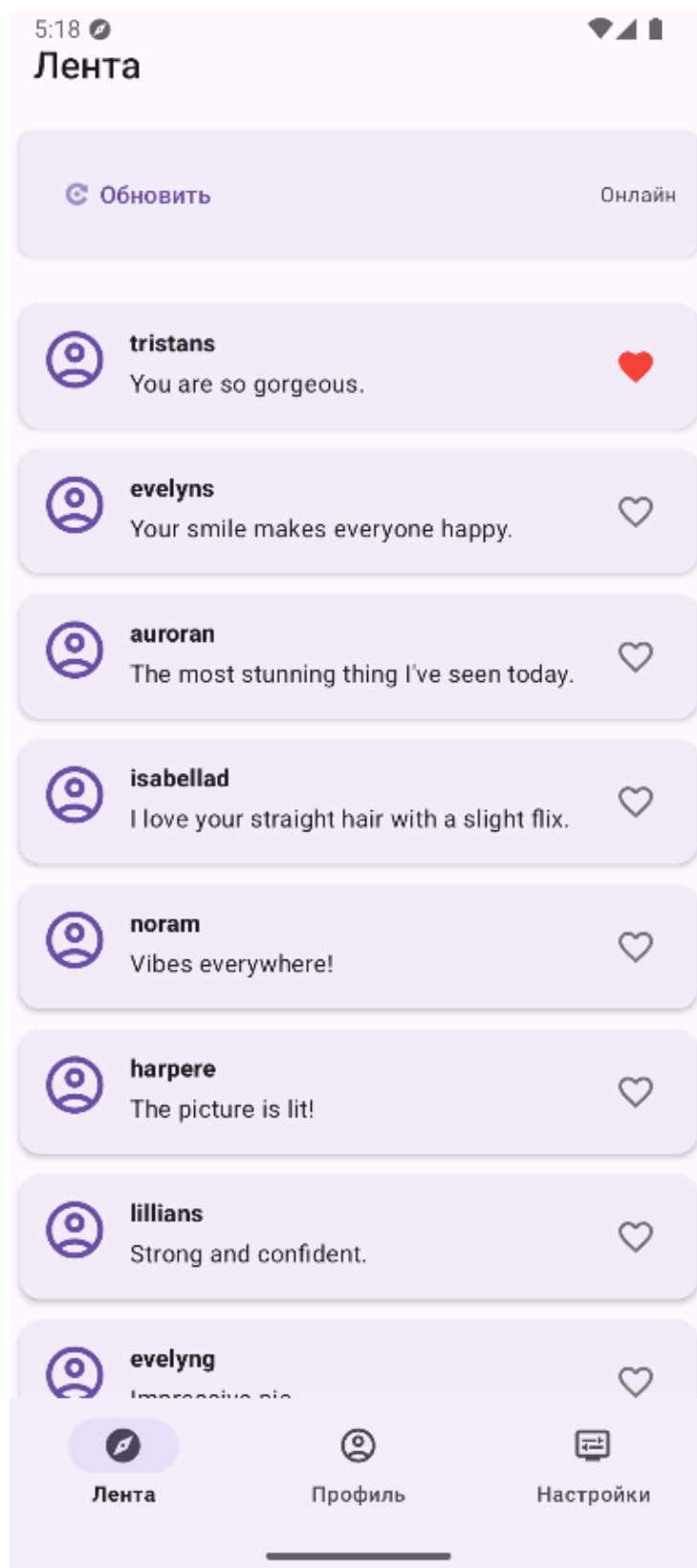


Рисунок 5 - Экран ленты сообщений с заполненным списком

## 4.5. Уведомления о новых сообщениях через WorkManager и Notification

Чтобы пользователь оперативно узнавал о появлении новых сообщений, приложение реализует систему локальных уведомлений. Задача решается периодическим опросом сервера в фоне и генерацией уведомления средствами Android.

**WorkManager для фоновой задачи.** Используется **WorkManager** – компонент, позволяющий планировать отложенные или периодические фоновые задачи, которые выполняются даже если приложение закрыто. Для этого создаётся класс, унаследованный от Worker:

```
class NewMessagesWorker(appContext: Context, workerParams: WorkerParameters)
    : Worker(appContext, workerParams) {
    override fun doWork(): Result {
        // 1. Проверяем наличие новых сообщений
        val newMessages = checkForNewMessages()
        // 2. Если есть новые сообщения, показываем уведомление
        if (newMessages.isNotEmpty()) {
            showNewMessagesNotification(newMessages.size)
        }
        return Result.success()
    }

    private fun checkForNewMessages(): List<Message> {
        // Логика проверки новых сообщений (например, запрос к API)
        // Можно запросить сообщения более новые, чем последний ID/время в
        // локальной базе
        // Здесь для примера: вызов репозитория
        val repository = ... // получить репозиторий или API
        return try {
            val latest = repository.fetchLatestMessages() // условный метод
            // Сравнить с локальными данными, выбрать новые
            latest.filter { it.timestamp > getLastKnownTimestamp() }
        } catch (e: Exception) {
            listOf()
        }
    }

    private fun showNewMessagesNotification(count: Int) {
        val notificationManager =
```

```

        applicationContext.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager
        // Создаем канал уведомлений (для Android 8.0+ необходимо)
        val channelId = "messages"
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                channelId, "Сообщения", NotificationManager.IMPORTANCE_DEFAULT
            )
            notificationManager.createNotificationChannel(channel)
        }
        // Формируем уведомление
        val notification = NotificationCompat.Builder(applicationContext,
channelId)
            .setSmallIcon(R.drawable.ic_message) // иконка уведомления
            .setContentTitle("Новые сообщения")
            .setContentText("У вас $count новых сообщений.")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .build()
        // Отправляем уведомление
        notificationManager.notify(1001, notification)
    }
}

```

В методе `doWork()` сначала выполняется некая проверка новых сообщений. В учебном примере мы имитируем эту логику методом `checkForNewMessages()`, который может внутри использовать уже настроенный репозиторий или API-сервис. Он возвращает список новых сообщений.

Если получен непустой список новых сообщений, вызываем `showNewMessagesNotification()`, который инициализирует **NotificationManager** и показывает уведомление:

- Создаем уведомление через `NotificationCompat.Builder`, указывая канал, иконку, заголовок и текст, в тексте кол-во новых сообщений.

- Метод `notify(id, notification)` показывает уведомление пользователю. Мы используем фиксированный ID (1001) для всех таких уведомлений, чтобы новое уведомление заменяло предыдущее

**Регистрация периодической работы.** Чтобы Worker запускался регулярно, нужно запланировать WorkRequest в приложении. Самое логичное место – в точке входа (например, в MainActivity при первом запуске). Мы используем PeriodicWorkRequest:

```
val workRequest = PeriodicWorkRequestBuilder<NewMessagesWorker>(15,
    TimeUnit.MINUTES)
    .setConstraints(
        Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED) // требование: интернет
            .build()
    )
    .build()
WorkManager.getInstance(context)
    .enqueueUniquePeriodicWork("check_new_messages",
        ExistingPeriodicWorkPolicy.KEEP, workRequest)
```

Данный код планирует повторяющуюся задачу каждые 15 минут. Constraint установлен как RequiredNetworkType.CONNECTED, чтобы WorkManager выполнял задачу только при наличии подключения к интернету. Мы используем **enqueueUniquePeriodicWork** с именем задачи "check\_new\_messages" и политикой KEEP – чтобы не создавать дубликатов при каждом запуске приложения.

После этого WorkManager берет на себя выполнение: примерно раз в 15 минут он будет создавать фонового работника NewMessagesWorker. Тот, в свою очередь, проверит сервер и при необходимости – отправит уведомление.

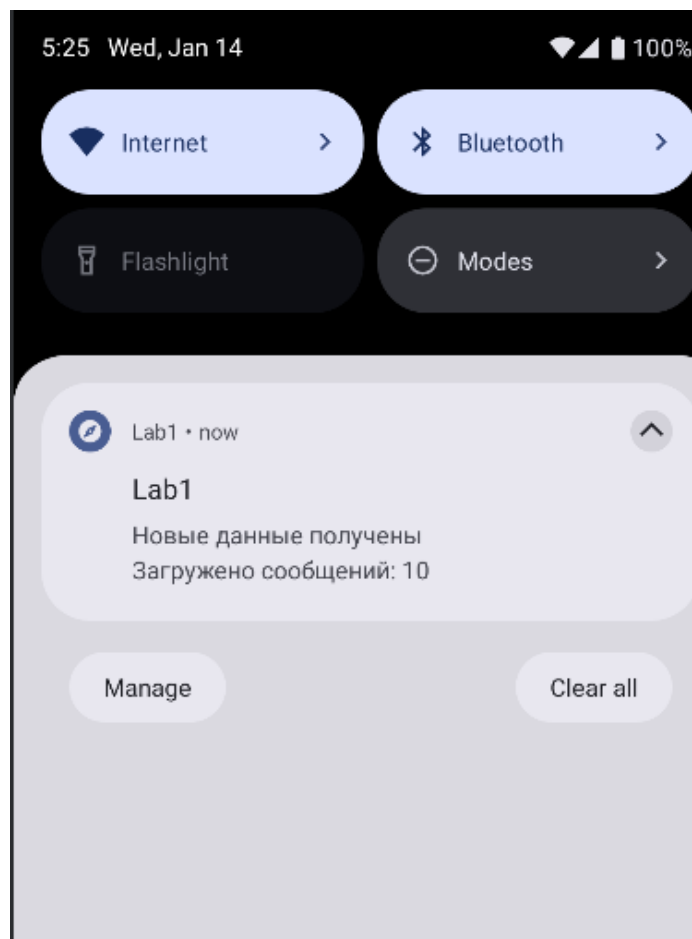
**Разрешения для уведомлений (Android 13).** Начиная с Android 13 (API 33) введено разрешение POST\_NOTIFICATIONS, требующее явного запроса у пользователя на показ уведомлений. Наше приложение должно учесть это: чтобы уведомления работали, надо запросить у пользователя разрешение. Мы реализовали это в Launch Activity (MainActivity) при первом запуске:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    if (checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS) !=
        PackageManager.PERMISSION_GRANTED) {
```

```
requestPermissions(arrayOf(Manifest.permission.POST_NOTIFICATIONS),  
REQ_CODE_NOTIF_PERMISSION)  
}  
}
```

Диалог запроса отобразится пользователю, и он может разрешить или отклонить. Если разрешено – уведомления будут приходить. Если отклонено – WorkManager всё равно будет выполняться, но notificationManager.notify() просто не покажет ничего (система блокирует).

Таким образом, для пользователей Android 13 наше приложение запросит разрешение на уведомления при первом запуске, чтобы реализовать функциональность оповещения о новых сообщениях.



**Рисунок 6 - Пример уведомления.**



## 5. Выводы

В ходе выполнения курсовой работы был разработан полноценный прототип Android-приложения – клиентского приложения для получения и отображения сообщений (аналог мессенджера или соцсети). Приложение реализовано с использованием современных технологий Android-разработки:

- Освоена и применена архитектура MVVM, позволяющая разделить логику приложения на слои Model, ViewModel и View.
- Использованы компоненты библиотеки Android Jetpack: LiveData, ViewModel для управления данными и жизненным циклом; Navigation Component для упрощённой навигации между экранами; Room для работы с базой SQLite; WorkManager для фоновых задач; NotificationCompat для пользовательских уведомлений
- Реализована навигация с помощью Navigation Graph и BottomNavigationView
- Созданы экраны Профиля и Настроек
- С помощью **WorkManager** и системы **Notification** реализована отправка локальных уведомлений о новых сообщениях.

**Освоенные технологии:** В процессе работы были освоены следующие технологии и подходы:

- Android Jetpack Architecture Components: **ViewModel**, **LiveData**, **Room**, **WorkManager**, **Navigation Component**, **Data Binding**
- Библиотека **Retrofit** для выполнения сетевых запросов и парсинга JSON, а также использование связки Retrofit + Gson.
- Основы работы с **SQLite** через библиотеку Room
- Работа с UI-элементами Material Design (BottomNavigationView, RecyclerView, etc.) и поддержка разных конфигураций (тёмная/светлая тема).

- Механизмы **планирования задач** (WorkManager) и **уведомления пользователей** (NotificationCompat, NotificationChannel).

- Система **runtime-разрешений** Android:

- Обработка **исключительных ситуаций**: отсутствие сети, ошибки API.

В результате выполнения проекта был получен ценный опыт разработки Android-приложения “полного цикла” – от проектирования архитектуры до реализации UI и фоновых сервисов. Приложение продемонстрировало работу основных компонентов Android.